

Contents

SVM ASIC — Design Rationale	1
1. Batch Size: 1000 Beats	1
2. Multi-Scale Feature Vector: 1-Beat, 10-Beat, 100-Beat Morphology	2
3. IS61WV51216 Async SRAM Selection	2
4. Distance Accumulator: 2 Drain Cycles	3
5. Why m3 Did Not Provide a Correct Implementation	4
6. Interface Evolution: SPI -> QSPI -> Wishbone	4
7. On-Chip SRAM Macros: Why They Were Abandoned	5
8. Other Key Decisions	6
9. Clock Frequency: 40 MHz (not 50 MHz)	7

SVM ASIC — Design Rationale

Project: 5-Class Cardiac Arrhythmia Classifier, sky130A

Author: Adam Handwerger . handwerg@pdx.edu

Course: ECE410, Portland State University

Date: 2026-06-04

This document records why each major architectural decision was made. Where an alternative was seriously considered, both the rejected option and the reason for rejection are noted.

1. Batch Size: 1000 Beats

The 1000-beat batch was chosen to satisfy three simultaneous constraints.

Clinical capture window. Paroxysmal arrhythmias (AFib, VT, SVT) can occur in short bursts and then resolve. A 24-hour Holter monitor is the clinical gold standard precisely because short monitoring windows miss intermittent events. At a resting heart rate of 80 bpm, 1000 beats spans 12.5 minutes — long enough to capture most paroxysmal episodes that last more than a few minutes, while short enough that the MCU can offload the full batch to the ASIC before the next batch begins.

SRAM capacity. The IS61WV51216 is a 512K x 16-bit device, giving 1 MB total. The SV matrix ($500 \times 256 \times 2 \text{ B} = 256 \text{ KB}$) occupies rows 0–499. The input matrix at 1000 beats x 256 features x 2 B = 512 KB occupies rows 500–1499. Total: 768 KB, well within the 1 MB device. A 2000-beat batch would not fit. A 500-beat batch would fit but halves the capture window.

MCU sleep budget. The MCU must stay active during feature extraction (250 Hz ECG) but can sleep during ASIC classification. At LAT=3 (IS61WV51216), classifying 1000 beats takes $1000 \times 9.7 \text{ ms} = 9.7 \text{ seconds}$. The ASIC uses 66 mW during this window; the MCU can be in deep sleep. For 1000-beat batches at 80 bpm the MCU sleeps 77% of the time during classification, which is a meaningful fraction of the 1.04 mW full-system budget.

ECG beats missed during sleep (accepted trade-off). The analog ECG frontend runs continuously regardless of MCU state, but the MCU cannot sample the ADC or extract features while in deep sleep. During the 9.7-second classification window, approximately 13 beats ($9.7 \text{ s} \times 80 \text{ bpm} / 60$) are not captured. This is an accepted loss: 13 missed beats represent 1.3% of

the 1000-beat window, and the 12.5-minute collection window already provides sufficient temporal coverage to detect paroxysmal arrhythmias that last more than a few minutes. A more complete solution would use a DMA controller or a dedicated low-power co-processor to buffer ECG samples into a ring buffer while the MCU sleeps, eliminating dropped beats entirely — but this adds hardware complexity that was out of scope for this design.

2. Multi-Scale Feature Vector: 1-Beat, 10-Beat, 100-Beat Morphology

The final 256-dimensional feature vector is:

Slice	Dims	Timescale
Single-beat morphology (+/-64 samples, amplitude-normalized)	128	~0.5 s
10-beat mean morphology template (mean of +/-32-sample windows)	64	~7.5 s
RR-interval history (99 intervals normalized to 308 ms reference)	64	~75 s

The three-timescale design was not chosen from first principles — it was forced by accuracy evidence.

Single-beat only (128-dim) achieved ~89%. The primary failure modes were VT/SVT confusion (both produce wide QRS in a single beat) and PVC/AFib confusion (a PVC during AFib looks morphologically similar to a sinus-rhythm PVC).

Adding 10-beat mean (+64 dims) raised accuracy to ~93%. The mean template captures whether the beat morphology is persistent (VT) or isolated (PVC). A sustained tachycardia pattern in the mean is a strong discriminant for VT vs. an isolated aberrant beat.

Adding RR-interval history (+64 dims) raised accuracy to 97.67%. The decisive feature for AFib is RR irregularity over many beats. A single-beat window cannot see this. The 99-interval history (normalized to the patient’s reference RR of 308 ms) provides the irregularity signature needed. SVT vs. VT discrimination also benefits from the RR regularity pattern: SVT has a sudden-onset regular tachycardia signature in the RR history; VT has a more gradual acceleration.

The feature convention follows de Chazal et al. (TBME 2004, 2006), which established these three slices as the minimum sufficient set for 5-class MIT-BIH classification.

3. IS61WV51216 Async SRAM Selection

The IS61WV51216 (ISSI, 512K x 16, 10 ns access, 3.3 V/5 V, 70-TSOP-II) was chosen over three alternative memory technologies.

vs. synchronous SRAM (e.g., CY7C1041): Synchronous SRAM requires a shared clock between the ASIC and the memory device. Routing a 40 MHz clock to an off-chip SRAM on a wearable PCB

introduces clock-to-PCB-edge jitter and demands matched trace lengths. Asynchronous SRAM is controlled purely by address and chip-enable timing — no shared clock, no skew budget, simpler PCB layout.

vs. Flash/NOR: Flash has read latencies of 60–100 ns and requires page-erase cycles for writes. Loading the SV matrix at boot takes ~100 ms (acceptable) but re-loading input features beat by beat would be far too slow.

vs. SPI PSRAM (e.g., ESP-PSRAM64): SPI PSRAM serializes 16-bit words over 4 lines, giving effective read bandwidth of ~80 Mbits/s — 8x slower than the 640 Mbits/s parallel bus. At 3.23 ms per beat (LAT=1) the parallel bus is already the bottleneck; SPI PSRAM would increase beat time to ~25 ms per beat.

vs. sky130A on-chip SRAM macros: This was the original plan. sky130A SRAM macros in the 256 KB range occupy approximately 1.5–2.0 mm² each. The 2500 x 2500 umm die is 6.25 mm². Two macros (one for SVs, one for input) would consume 3–4 mm² of the 6.25 mm² die, leaving less than 40% for the compute datapath. P&R routing becomes infeasible. Moving storage off-chip freed the entire die for logic and left 86% of the die utilization headroom.

RAM_LATENCY=3 vs. LAT=2: The IS61WV51216 datasheet specifies 10 ns access time, which technically fits within one 25 ns clock cycle. LAT=3 was chosen to provide margin for: (1) PCB trace propagation delays between ASIC GPIO pins and SRAM data pins, (2) input flip-flop setup time inside the ASIC, and (3) SRAM access-time derating at elevated temperature and reduced supply voltage in field conditions. LAT=2 would likely work on a benchtop at 25 degC but is not safe over the full operating envelope.

4. Distance Accumulator: 2 Drain Cycles

The distance accumulator pipeline is two stages deep:

```
Stage 1: diff_sq = (x[i] - sv[i])^2    (registered, 1-cycle latency)
Stage 2: acc      = acc + diff_sq      (registered, 1-cycle latency)
```

After the 256th input pair is presented (`valid_in` asserted for cycle 256), two more clock cycles must pass before the final accumulation result is valid in `acc`:

- Cycle 257: `diff_sq` for the 256th pair propagates to the adder input
- Cycle 258: the adder result for the 256th pair propagates into `acc`

These two cycles are the “drain” — the pipeline is flushed of the last two in-flight values.

Without the drain, the FSM would latch the distance result one or two cycles early, silently dropping the last one or two feature dimensions. The effect is a systematic underestimate of squared distance for all SV comparisons, which narrows the effective RBF kernel width and shifts decision boundaries — producing classification errors that do not manifest as detectable flags.

The drain cycles were discovered during m3 testbench development. The distance-zero unit test (`tb_dist_zero.sv`) checks that `K(x, sv)` outputs exactly 1024 ($\exp(0) = 1.0$ in Q6.10) when `x = sv`. Without the drain, this test would fail on the 256th-dimension comparison because the accumulator would report a small nonzero distance instead of zero, pushing the kernel output below

1024. The fix — holding `valid_in` low for 2 extra cycles after the 256th pair before reading the accumulator — is tested explicitly in the pipeline.

5. Why m3 Did Not Provide a Correct Implementation

Milestone 3 targeted RTL verification and synthesis only (no place-and-route). It reported 98.67% accuracy and 19/19 tests passing. However, the m3 architecture had three problems that made it unsuitable for the Caravel silicon target.

Problem 1: Decision function computed by the MCU, not the ASIC.

In m3, the compute core output raw kernel values $K(x, sv_i)$ per SV to an off-chip workspace RAM. The host MCU read these values after `done` and computed the OVR decision function ($\sum \alpha_i K(x, sv_i) + \text{bias}$) in software. This required: - 500 KB of off-chip workspace RAM for per-SV kernel outputs - The MCU to stay active and perform 500 multiply-accumulate operations per class per beat - An additional SRAM interface beyond the SV RAM

The result was that the ASIC accelerated only the kernel evaluation, not the classification step — the most numerically expensive part ($\sum \alpha_i K$) was still done in software. The m4/m5 architecture moved the alpha table on-chip (`alpha_table[500]`) and the accumulation into the `COMPUTE_KERNEL` FSM state, so the ASIC outputs a 3-bit class label and the MCU reads a single STATUS register.

Problem 2: On-chip FIFO was incompatible with sky130A area constraints.

m3 specified an 8192-word (16 KB) synchronous FIFO for QSPI input buffering. Implementing 16 KB on-chip in sky130A requires either a large FF-based register array (8192 x 16 flip-flops ~ 130K standard cells) or a sky130A SRAM macro. The smallest practical sky130A SRAM macros occupy 1.5–2.0 mm² — a large fraction of the 6.25 mm² die budget — and the OpenLane flow for sky130A does not support custom SRAM macro instantiation without significant harness changes. An FF-based FIFO of this size would by itself exceed the target cell count for the entire core. The m4 architecture replaced the FIFO with a 256-word on-chip register buffer (512 B, FF-based), reading directly from off-chip SRAM one word per cycle.

Problem 3: QSPI streaming forced the MCU to stay awake during classification.

m3 used a QSPI interface to stream feature data beat-by-beat from the MCU to the FIFO. This meant the MCU had to remain active — driving QSPI, monitoring FIFO backpressure, and capturing kernel outputs — for the entire classification session. The duty-cycle power savings of the batch architecture (MCU sleeps while ASIC classifies) were not achievable in the m3 model. See Section 6 for the full interface evolution.

6. Interface Evolution: SPI -> QSPI -> Wishbone

Three interface architectures were designed and discarded before reaching the final Wishbone + off-chip SRAM model.

SPI (v4–v5): Standard 4-wire SPI, one data line. At 40 MHz, effective bandwidth is 5 MB/s. A 512-byte feature vector takes 102 μ s to stream. This was acceptable for single-beat operation but could not sustain 1000-beat batch throughput — the MCU would spend 102 ms/batch just on data transfer, with no time to sleep.

QSPI (v6–v7): Quad SPI, 4 data lines simultaneously, quadrupling bandwidth to 20 MB/s. Feature vector load time dropped to ~25 μ s. However, three problems emerged: 1. Caravel’s management SoC provides limited bidirectional GPIO control. Achieving compliant QSPI IO-turnaround timing (MOSI->MISO direction switch) required bit-banging from the RISC-V core, introducing jitter that violated the QSPI setup window at 40 MHz. 2. A compliant QSPI peripheral required a full protocol state machine in the wrapper, consuming area and introducing chip-select and dummy-cycle edge cases that complicated testbench integration. 3. QSPI still streamed data beat-by-beat, forcing the MCU to stay active throughout classification and negating duty-cycle power savings.

Wishbone + off-chip SRAM (v8–v9, final): The final architecture abandoned streaming input entirely. The MCU pre-loads the full SV matrix and input feature matrix into a single IS61WV51216 off-chip SRAM via a simple parallel write (no protocol, no ASIC involvement). The ASIC then reads both matrices autonomously via a 19-bit address bus (GPIO[28:10]) and 16-bit data bus (Logic Analyzer LA[15:0]), classifying all 1000 beats without any further host interaction. The Wishbone B4 bus — provided natively by the Caravel management SoC — is used only for the handful of configuration register writes (ALPHA_WR, NUM_SAMPLES, CONTROL[start]) and the STATUS poll. This model: - Eliminates streaming bandwidth constraints - Allows the MCU to sleep for the full 9.7-second classification window - Uses the Caravel-standard interface, reducing wrapper area and simplifying DV

7. On-Chip SRAM Macros: Why They Were Abandoned

The original plan (m3 era, pre-P&R) assumed the SV matrix could be stored in an on-chip sky130A SRAM macro, with the input FIFO as an FF-based register array or a second macro.

sky130A SRAM macro constraints. Available sky130A SRAM macros from the PDK (sky130_sram_1kbyte_1rw1r through sky130_sram_4kbyte_1rw1r, and the community DFFRAM generator) scale poorly above ~4 KB. The SV matrix at 256 KB would require either a non-standard macro size (not in the sky130A PDK) or tiling 64 x 4 KB macros — each with its own power rails, address decoder, and timing closure requirements. OpenLane’s P&R flow does not handle multi-macro tiling automatically; each macro placement must be manually specified in the floorplan.

Area budget. Even a single 256 KB macro would occupy approximately 1.5–2.0 mm^2 on sky130A. The 2500 x 2500 μm die is 6.25 mm^2 . With two macros (SV + input), 3–4 mm^2 of the die would be consumed by storage, leaving less than 2 mm^2 for the compute datapath. The compute core (distance accumulator, Horner LUT, alpha register file, FSM) synthesized to approximately 14% utilization (0.875 mm^2 equivalent) — it would not fit alongside two 256 KB macros.

FF-based FIFO alternative. Implementing the 16 KB input FIFO as an FF-based shift register (8192 x 16-bit flip-flops) avoids the macro problem but creates a different one: 130K flip-flops in the input FIFO alone would exceed the target cell count for the entire core (146K). The FIFO would dominate the die and create a congestion nightmare for P&R.

Resolution: move all large storage off-chip. The off-chip IS61WV51216 absorbs all storage requirements (SVs + input matrix, up to 768 KB). On-chip, the only large structure is the alpha_table[500] register file (8 KB, ~80K standard cells), which is accessed at high frequency during kernel summation and cannot be moved off-chip without adding another SRAM bus. The on-chip Horner LUT (256 x 16 = 4 KB) and input feature buffer (256 x 16 = 512 B) are small enough to infer as flip-flop arrays within the normal cell budget.

8. Other Key Decisions

8.1 Support Vector Count: 250 -> 500

The initial design used 50 SVs per class (250 total). Cross-validation on MIT-BIH reached ~94–95% — below the sklearn float baseline of 97.67%. Increasing to 100 SVs per class (500 total) closed the accuracy gap to zero. The cost was doubling the `alpha_table` register file (4 KB -> 8 KB), increasing cell count from ~80K to ~146K and utilization from 7% to 14%. The 500-SV design still closes timing cleanly (+7.83 ns setup slack) and remains well within the die area budget.

8.2 FP16 -> Q6.10 Precision

An intermediate design used IEEE 754 FP16 (half-precision). FP16 has a 5-bit exponent with a maximum representable value of 65,504. The distance accumulator sums 256 squared differences; if the sum exceeds 65,504, FP16 saturates to infinity, producing $\exp(-\infty) = 0$. This silent saturation was observed on ~3% of test samples, causing misclassifications that did not appear in the float baseline.

Q6.10 was designed with the accumulator range in mind. The integer portion (6 bits, range ± 32) sizes exactly to the maximum expected squared distance for unit-normalized 256-dim features (~2048 at worst). Overflow is detected and reported as `ERR_DIST_OVERFLOW` (sticky, error code 0x4) rather than silently corrupting the kernel output. Q6.10 is also simpler to implement: no exponent handling, no NaN propagation, no subnormal-number cases.

8.3 Horner LUT: Range Reduction Required

The Horner polynomial for $\exp(x)$ converges in the range $[-1, 0]$. The kernel argument `-gamma x d^2` ranges from -2048 to 0 — far outside $[-1, 0]$. A direct Horner polynomial over this range requires degree 8 or higher for acceptable precision, and at the extremes the polynomial wraps in Q6.10 fixed-point, producing a near-random kernel output (this was observed in m3 v6 as ~20% accuracy collapse).

The fix is a two-stage evaluation: 1. Look up the coarse $\exp(-I)$ from a 256-entry table, where I is the integer portion of the scaled argument. The LUT spans $[-8, 0]$ in steps of $1/32$. 2. Evaluate a degree-3 Horner polynomial on the residual $-F$ (the fractional remainder), which lies in $[-1/64, 0]$. At this narrow range, degree 3 is accurate to 10-bit precision.

The LUT is $256 \times 16\text{-bit} = 4\text{ KB}$, stored in flip-flop registers on-chip. It is preloaded once at reset and never changes during inference.

8.4 OVR vs. OVO Classification

One-vs-rest (OVR) was chosen over one-vs-one (OVO) for two reasons:

1. **Five class accumulators vs. ten.** OVO requires $C(5,2) = 10$ binary classifiers with a voting scheme; OVR requires 5. Each classifier requires its own alpha weights. With OVR, the `alpha_table` is 500 entries (100 per class $\times$ 5 classes); with OVO it would be up to 1000 entries (100 per pairing $\times$ 10 pairings), doubling the register file.

2. **Argmax is a single comparator.** OVR decision: the class with the highest accumulated score $\sum \alpha_i K(x, sv_i)$ wins. OVO decision requires a vote-counting circuit across 10 binary outputs. The argmax of 5 32-bit accumulators synthesizes to a small comparator tree; the OVO vote counter would require a 3-bit majority-logic circuit.

8.5 Output Race Condition: Per-Beat Result Buffer

The original design held the current beat's class label in a single `class_out[2:0]` GPIO register, overwritten at the start of each new beat. The MCU had to poll and capture the result within the window between `sample_rdy` assertion and the next beat beginning (~ 9.7 ms at LAT=3). In RTL cosimulation this window was reliably captured. In projected silicon (RISC-V core at variable CPI, possible IRQ latency), the window is not guaranteed.

A per-beat result buffer (rolling SRAM write of class labels indexed by beat number) was added so that the MCU can read the full batch result after `done` asserts, with no timing constraint on when it reads. This also enables post-hoc logging over BLE and supports a rolling window mode for continuous online display.

9. Clock Frequency: 40 MHz (not 50 MHz)

The design targets 40 MHz (25 ns clock period). The critical path runs through the Horner polynomial evaluation — a chain of Q6.10 multiply-accumulate operations that approximate $\exp(-\gamma D)$. Each step is `result = result * x + coefficient`; the 16x16-bit multiplier is the slowest element in the chain.

At TT/25 degC/1.80V, post-P&R STA gives WNS = +3.96 ns. The critical path therefore takes 25 - 3.96 = **21.04 ns**. The maximum achievable clock frequency on sky130 at TT is $1 / 21.04 \text{ ns} \sim$ **47.5 MHz**.

At 50 MHz (20 ns period), the same path has slack of 20 - 21.04 = **-1.04 ns** — a setup violation in the TT corner. The design would fail at its nominal operating point if clocked at 50 MHz, not just in the SS extreme corner.

Why 40 MHz rather than 47.5 MHz: The 3.96 ns margin provides headroom for process variation, PCB clock noise, and temperature excursion above 25 degC without entering the SS failure regime. At 40 MHz the batch completes in ~ 9.87 ms per beat, well within the 750 ms heartbeat window — throughput is not the bottleneck.

To reach 50 MHz the options are: 1. **Pipeline the Horner stages** — insert register cuts between multiply-add steps, halving combinational depth at the cost of one extra latency cycle per kernel. 2. **Switch to sky130_fd_sc_hs** — higher drive strength cells, faster paths, ~ 15 –20% larger area. 3. **Reduce Horner polynomial degree** — shorter critical path, less accurate kernel approximation near the fractional boundary.